

19 - Goal 系统详解

阅读时间：约 1 小时

前置知识：06 - 安装与上手

学习目标：理解 Nanobot Goal 系统的设计原理、`/goal` 命令机制、Sustained Goal 状态管理

目录

- 19.1 Goal 系统概述
 - 19.2 `/goal` 命令机制
 - 19.3 Sustained Goal 状态管理
 - 19.4 Goal vs 普通对话
 - 19.5 实战：使用 Goal 完成多步骤任务
 - 19.6 面试话术
-

19.1 Goal 系统概述

Goal 系统是 Nanobot v0.2.0 引入的核心功能，用于维持跨轮次的长期目标。

解决的问题：- 普通对话中，Agent 容易在多次迭代后忘记最初目标 - 复杂任务需要多步骤执行，需要持续跟踪进度 - 用户需要明确的目标管理和进度可视化

核心价值：- 通过 `/goal` 命令启动长期目标 - Goal 状态在会话间持久化 - WebUI 实时显示 Goal 进度 - 自动超时控制 (Wall timeout / LLM timeout)

19.2 `/goal` 命令机制

启动 Goal

You: `/goal` 帮我重构这个项目的代码结构

Agent: [启动 Goal 模式]

目标：重构项目代码结构

状态：Active

Goal 状态追踪

Goal 启动后，Agent 会：1. 在会话 metadata 中存储 `goal_state` 2. 每次迭代检查目标进度 3. 在 WebUI 显示实时进度 4. 完成后自动标记为 `completed`

源码实现

```
# nanobot/session/goal_state.py
```

```
GOAL_STATE_KEY = "goal_state"
```

```
def sustained_goal_active(metadata) -> bool:
    """ 检查是否有活跃的 Goal """
    goal = parse_goal_state(goal_state_raw(metadata))
    return isinstance(goal, dict) and goal.get("status") == "active"
```

```
def sustained_goal_turn(metadata, message_metadata) -> bool:
    """ 判断当前轮次是否使用 Goal 模式 """
    if sustained_goal_active(metadata):
        return True
```

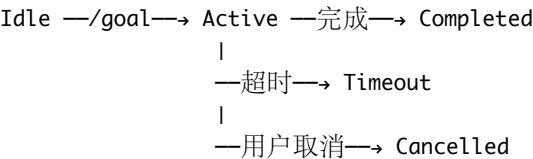
```
# /goal 命令触发
return str(message_metadata.get("original_command") or "").strip() == "/goal"
```

19.3 Sustained Goal 状态管理

Goal 状态结构

```
{
  "status": "active",
  "objective": " 重构项目代码结构",
  "ui_summary": " 正在分析模块依赖...",
  "turn_count": 5,
  "started_at": "2026-06-17T10:00:00Z"
}
```

状态转换



Runtime Context 注入

当 Goal 激活时，会在 System Prompt 中注入：

```
# nanobot/session/goal_state.py 第 73-89 行
def goal_state_runtime_lines(metadata):
    """Goal 激活时注入 Runtime Context 的行"""
    goal = parse_goal_state(...)
    if goal.get("status") == "active":
        objective = goal.get("objective")
        return [
            "Goal (active):",
            objective,
            f"Summary: {goal.get('ui_summary')}"
        ]
```

效果：每次 Agent 迭代都能看到当前目标，不会迷失方向。

19.4 Goal vs 普通对话

特性	普通对话	Goal 模式
目标持久化	不持久化	存储在 metadata
迭代限制	默认 40 次	可配置更长
超时控制	标准 timeout	Wall timeout = 0（禁用）
WebUI 显示	无特殊显示	实时进度条
适用场景	简单问答	多步骤复杂任务

超时控制差异

```
# nanobot/session/goal_state.py 第 109-126 行
def runner_wall_llm_timeout_s(...):
```

```

"""Goal 模式下的超时控制"""
if sustained_goal_turn(...):
    return 0.0 # 禁用 asyncio.wait_for
return None # 使用默认 NANOBOT_LLM_TIMEOUT_S

```

设计意图：Goal 任务可能需要长时间运行，不应被标准超时中断。

19.5 实战：使用 Goal 完成多步骤任务

场景：自动化代码审查

You: /goal 审查整个项目的代码质量

Agent: [Goal Active]

目标：审查整个项目的代码质量

[第1轮] 扫描项目结构...

[第2轮] 分析主要模块...

[第3轮] 检查代码规范...

[第4轮] 识别潜在 Bug...

[第5轮] 生成审查报告...

[Goal Completed]

审查完成，发现 3 个警告，5 个建议。

WebUI 中的 Goal 显示

在 WebUI 中，Goal 会显示为：- 顶部进度条：Goal：审查整个项目的代码质量 - 摘要信息：Summary：正在分析主要模块... - 实时更新：每次迭代后自动刷新

查看 Goal 状态

查看会话 metadata (包含 goal_state)

cat workspace/sessions/cli:direct.jsonl | python -m json.tool | grep -A 10 goal_state

19.6 面试话术

问题：Nanobot 的 Goal 系统是如何解决长任务迷失问题的？

“Nanobot 的 Goal 系统通过 /goal 命令启动长期目标模式。核心设计是：在会话 metadata 中存储 goal_state (包含目标、状态、摘要等信息)，每次 Agent 迭代时，通过 goal_state_runtime_lines 将当前目标注入到 Runtime Context 中，确保 Agent 始终知道最终目标。同时，Goal 模式禁用标准超时控制 (返回 0.0)，允许长时间运行。WebUI 会实时显示 Goal 进度，用户可以直观跟踪任务状态。这种设计解决了 Agent 在多次工具调用后容易迷失方向的问题。”

问题：Goal 系统和普通对话的技术差异是什么？

“主要有三个差异：第一，Goal 状态持久化在会话 metadata 中，通过 goal_state key 存储，而普通对话没有这个状态；第二，Goal 模式通过 sustained_goal_turn 函数判断，会注入 Runtime Context 并禁用超时控制；第三，WebUI 有专门的 Goal 显示逻辑，通过 goal_state_ws_blob 获取实时进度。这些差异使得 Goal 模式适合多步骤复杂任务，而普通对话适合简单问答。”

扩展阅读：18 - WebUI 实战 ——WebUI 中的 Goal 进度显示